



Тема 3.8. Разбор логов (1 ак. ч.)

Теория (0,3 ч.): анализ ошибок, stack trace, корреляция событий, поиск первопричин. Практика (0,7 ч.): локализация проблемы по логам, генерация гипотез и шагов воспроизведения.

Тема 3.9. Генерация и проверка гипотез при отладке (1 ак. ч.)

Теория (0,3 ч.): формулирование проверяемых гипотез, приоритизация и отделение симптома от первопричины. Практика (0,7 ч.): экспериментальная проверка гипотез, регрессионный тест и минимальное исправление.

1. Что мы делаем в заключительной работе

Сейчас я проведу полноценное небольшое расследование дефекта. Это будет не задача «найди ошибку в этой функции», где место проблемы уже фактически известно.

У меня будет баг-репорт от тестировщика, лог нескольких запросов, stack trace и существующий код проекта. Сначала я дам Hermes только описание инцидента и лог. После этого разрешу ему исследовать код. Затем агент сформирует несколько проверяемых гипотез, расположит их по приоритету и для каждой предложит способ проверки и ожидаемый результат.

После этого мы действительно выполним воспроизведение. Когда первопричина будет подтверждена фактическими данными, создадим regression-тест, убедимся, что он падает на дефектной реализации, выполним минимальное исправление и повторно запустим весь сценарий.

Это заключительная практика, поэтому здесь мы связываем несколько навыков, которые уже использовали раньше: работу агента с файлами, терминалом, Git, тестами, документацией и контролируемым изменением кода.

2. Используем существующий проект

Открываю PowerShell в корне существующего проекта `hermes-multiagent` и проверяю состояние:

```
```powershell id="7gkpzl" git status --short --branch
```

Перед заключительной работой мне нужна понятная контрольная точка: изменения предыдущих практик должны быть отделены от текущего расследования. Способ организации веток и коммитов выбирается в соответствии с принятым в проекте процессом Git.

## 3. Создаём отдельную область финального инцидента

Создаю каталог:

```
```powershell id="ytqz5b"
New-Item -ItemType Directory -Force final_debug_lab
```

Создаю каталог логов:

```
```powershell id="g0u4vn" New-Item -ItemType Directory -Force final_debug_lab\logs
```

Рабочие журналы не должны попадать в историю Git. Добавляю в `.gitignore` правило:

```
```gitignore id="4m96xa"
/final_debug_lab/logs/*.log
```

Создаю пустой пакет:

```
```powershell id="2m3lxm" New-Item -ItemType File -Force final_debug_lab__init__.py
```

Мы не будем ломать код предыдущей практики специально. Новый каталог будет имитировать небольшую подсистему, которая использует уже созданный `testing\_lab`.

## 4. Проверяем инструменты Hermes

Открываю корень проекта в Hermes и создаю новую сессию.

Проверяю:

```
```text id="3oekwk"
/tools list
```

Для расследования нужны инструменты чтения и поиска файлов, а также терминал. Их названия и объединение в наборы зависят от установленной версии и конфигурации, поэтому проверяю фактически доступные возможности в текущей сессии. Если какого-либо инструмента нет, включаю подходящий файловый или терминальный набор в интерфейсе используемой среды.

5. Создаём источник заказов

Теперь создаю файл:

```
```text id="b3hbsc" final_debug_lab\legacy_orders.py
```

Содержимое:

```
```python id="8s3vca"
from copy import deepcopy
from datetime import date
from decimal import Decimal
```

```
_ORDERS = {
    1001: {
```

```

        "id": 1001,
        "status": "delivered",
        "delivered_at": date(2026, 9, 10),
        "items": [
            {
                "product_id": 101,
                "quantity": 2,
                "unit_price": Decimal("100.00"),
                "discount_percent": Decimal(10),
            }
        ],
    },
    1002: {
        "id": 1002,
        "status": "delivered",
        "delivered_at": "2026-09-10",
        "items": [
            {
                "product_id": 101,
                "quantity": 2,
                "unit_price": Decimal("100.00"),
                "discount_percent": Decimal(10),
            }
        ],
    },
}

def get_order(order_id: int) -> dict:
    """Возвращает независимую копию заказа из legacy-источника."""

    if order_id not in _ORDERS:
        raise KeyError(f"Заказ {order_id} не найден")

    return deepcopy(_ORDERS[order_id])

```

Обратите внимание: я пока ничего не объясняю студентам про различие двух записей.

Для нас это просто источник двух заказов.

6. Создаём программу воспроизведения

Теперь создаю:

```
``text id="9p35zu" final_debug_lab\reproduce_bug.py
```

Код:

```
```python id="c374ae"
import logging
import sqlite3
import time
from datetime import date
from pathlib import Path

from final_debug_lab.legacy_orders import get_order
from testing_lab.refunds import RefundService, ReturnRepository

LOG_DIR = Path(__file__).parent / "logs"
LOG_FILE = LOG_DIR / "app.log"
LOG_RETENTION_DAYS = 30

def _remove_expired_logs() -> None:
 """Удаляет диагностические журналы старше установленного срока."""

 cutoff = time.time() - LOG_RETENTION_DAYS * 24 * 60 * 60

 for log_file in LOG_DIR.glob("*.log"):
 if log_file.stat().st_mtime < cutoff:
 log_file.unlink()

def configure_logging() -> logging.Logger:
 """Настраивает консольный и файловый журналы воспроизведения."""

 LOG_DIR.mkdir(parents=True, exist_ok=True)
 _remove_expired_logs()

 logger = logging.getLogger("final_debug_lab")
 logger.setLevel(logging.INFO)

 for handler in logger.handlers:
 handler.close()
 logger.handlers.clear()

 formatter = logging.Formatter("%(asctime)s %(levelname)s %(message)s")
 file_handler = logging.FileHandler(LOG_FILE, mode="w", encoding="utf-8")
 console_handler = logging.StreamHandler()
 file_handler.setFormatter(formatter)
 console_handler.setFormatter(formatter)
 logger.addHandler(file_handler)
 logger.addHandler(console_handler)

 return logger

def process_return(
 logger: logging.Logger,
```

```
request_id: str,
order_id: int,
) -> None:
 """Выполняет один сценарий возврата и записывает его результат."""

 logger.info(
 "request_id=%s событие=заявка_получена заказ=%s",
 request_id,
 order_id,
)

 order = get_order(order_id)
 logger.info(
 "request_id=%s событие=заказ_загружен статус=%s delivered_at=%r",
 request_id,
 order["status"],
 order["delivered_at"],
)

 connection = sqlite3.connect(":memory:")
 repository = ReturnRepository(connection)
 service = RefundService(repository)

 try:
 result = service.create_request(
 order=order,
 requested={101: 1},
 today=date(2026, 9, 20),
)
 logger.info(
 "request_id=%s событие=возврат_создан return_id=%s сумма=%s статус=%s",
 request_id,
 result["id"],
 result["amount"],
 result["status"],
)
 except Exception:
 logger.exception(
 "request_id=%s событие=ошибка_возврата заказ=%s",
 request_id,
 order_id,
)
 finally:
 connection.close()

def main() -> None:
 """Воспроизводит успешный и ошибочный сценарии возврата."""

 logger = configure_logging()
 logger.info("событие=начало_воспроизведения")
```

```
process_return(logger, request_id="REQ-OK-1001", order_id=1001)
process_return(logger, request_id="REQ-BUG-1002", order_id=1002)
logger.info("событие=конец_воспроизведения")
```

```
if __name__ == "__main__":
 main()
```

## 7. Создаём баг-репорт

Теперь создаю:

```
```text id="ujvmk7" final_debug_lab\BUG_REPORT.md
```

Содержимое:

```
```markdown id="f434dc"
```

```
BUG-001. Ошибка при создании возврата для старого заказа
```

```
Окружение
```

Локальное учебное Python-окружение проекта.

```
Наблюдаемая проблема
```

При создании заявки на возврат для некоторых старых заказов операция завершается ошибкой. Новые заказы с аналогичными товарами обрабатываются нормально.

```
Пример
```

Для заказа 1001 заявка на возврат создаётся. Для заказа 1002 при тех же параметрах возврата возникает ошибка.

```
Ожидаемый результат
```

Для обоих доставленных заказов должна создаваться заявка на возврат со статусом `requested`.

```
Фактический результат
```

Заказ 1001 обрабатывается успешно. Заказ 1002 завершается ошибкой.

```
Воспроизводимость
```

Для заказа 1002 проблема воспроизводится при каждом запуске.

```
Дополнительная информация
```

Заказ 1002 относится к данным, перенесённым из старой версии системы. Точная причина ошибки неизвестна.

Это тот баг-репорт, который получает наша команда.

Причину я туда не записываю.

## 8. Создаём фактический лог

Теперь запускаю программу сам:

```
```powershell id="o6g32t" ..env\Scripts\python.exe -m final_debug_lab.reproduce_bug
```

После выполнения должен появиться:

```
```text id="wfk7h"
final_debug_lab\logs\app.log
```

Открываю его:

```
```powershell id="7oy1gc" Get-Content .\final_debug_lab\logs\app.log
```

Один запрос должен завершиться успешно.

Второй должен содержать stack trace.

Сохраняю исходное свидетельство отдельно, потому что следующий запуск перезапишет `app.log`:

```
```powershell id="d2q8ck"
Copy-Item .\final_debug_lab\logs\app.log .\final_debug_lab\logs\before_fix.log -Force
```

На этом этапе я не начинаю сразу искать строку в исходном коде. Сначала хочу продемонстрировать нормальный процесс расследования.

## 9. Проверяем сохранённое свидетельство

Убеждаюсь, что снимок содержит ошибочный запрос и трассировку:

```
```powershell id="nrr4p4" Get-Content .\final_debug_lab\logs\before_fix.log
```

Также проверяю, что рабочие журналы исключены из Git:

```
```powershell id="tw41b"
```

```
git check-ignore final_debug_lab\logs\app.log final_debug_lab\logs\before_fix.log
```

Контрольной точкой расследования служит неизменяемый снимок `before_fix.log`. Коммит будет создан один раз после завершения всей практики.

## 10. Первый промпт Hermes: только баг-репорт и лог

---

Теперь перехожу в Hermes.

Первый промпт специально запрещает читать исходный код:

```
``text id="7l6ov1" Мы расследуем баг BUG-001.
```

Сначала работай только с:

final\_debug\_lab/BUG\_REPORT.md

и:

final\_debug\_lab/logs/before\_fix.log

Исходный код пока не исследуй.

Ничего не изменяй.

Проанализируй баг-репорт и лог.

Сначала раздели информацию на:

подтверждённые факты;

наблюдаемый симптом;

непосредственную техническую ошибку;

пока ещё неизвестную первопричину.

Затем восстанови последовательность событий отдельно для:

REQ-OK-1001

и:

REQ-BUG-1002.

При анализе stack trace укажи:

тип исключения;

файл;

функцию;

операцию, на которой программа завершилась ошибкой.

---

Не объявляй последнюю строку `stack trace` первопричиной автоматически.

После анализа остановись.

Не предлагай исправление.

Это первый реальный промпт практики.

## 11. Что я проверяю в ответе агента

Мне важно, чтобы Hermes не написал сразу:

```
```text id="dhnyje"
```

Причина найдена, нужно изменить такую-то строку.

Пока он видел только баг-репорт и лог.

Он может установить тип исключения и место, где программа упала. Это факт.

Но root cause ещё требует проверки исходного состояния данных и кода.

Я отдельно смотрю, связал ли агент строки:

```
```text id="4tah0t" REQ-OK-1001
```

между собой и отдельно:

```
```text id="wtzrzi"
```

REQ-BUG-1002

Так мы демонстрируем корреляцию событий по `request_id`.

12. Теперь разрешаем изучить код

Отправляю второй промпт:

```
```text id="c2fp39" Теперь разрешаю исследовать исходный код.
```

Прочитай только файлы, которые необходимы для объяснения `stack trace` и различия между успешным и неуспешным запросом.

В первую очередь исследуй:

`final_debug_lab/legacy_orders.py`

`final_debug_lab/reproduce_bug.py`

`testing_lab/refunds.py`

Также можешь прочитать:

testing\_lab/CONTRACT.md

Ничего не изменяй.

После чтения не исправляй код.

Сопоставь:

баг-репорт;

лог;

stack trace;

успешный запрос REQ-OK-1001;

ошибочный запрос REQ-BUG-1002;

исходный код.

После этого сформируй несколько возможных гипотез первопричины.

Каждая гипотеза должна быть проверяемой.

Не создавай гипотезы, которые не связаны с имеющимися свидетельствами.

Теперь Hermes получает дополнительный контекст.

## 13. Третий промпт: заставляем агента оформить гипотезы правильно

Следующий запрос нужен именно для темы 3.9.

Отправляю:

```text id="sy77ds"

Теперь оформи диагностические гипотезы.

Ограничься максимум четырьмя гипотезами.

Для каждой гипотезы укажи:

формулировку возможной причины;

какие факты её поддерживают;

какие факты ей противоречат или пока отсутствуют;

приоритет проверки: высокий, средний или низкий;

конкретное действие для проверки;

ожидаемый результат, если гипотеза верна;

ожидаемый результат, если гипотеза неверна.

Приоритет определяй по двум критериям:

насколько хорошо гипотеза объясняет имеющиеся данные;

насколько быстро и безопасно её можно проверить.

Пока ничего не запускай и не изменяй.

После плана остановись.

Вот такой формат уже превращает AI-ответ из набора догадок в диагностический план.

14. Какую гипотезу я сам считаю основной

Теперь я отдельно показываю студентам своё рассуждение.

В успешном запросе используется заказ 1001.

В неуспешном запросе используется заказ 1002.

Статус у обоих:

```
```text id="5gyxkz" delivered
```

Товары одинаковы.

Дата календарно тоже соответствует одному и тому же дню.

Но в логе представление ``delivered_at`` отличается.

У одного заказа мы видим объект даты.

У другого значение пришло в другой форме.

Stack trace указывает на вычисление количества дней между:

```
```python id="xfpq3r"
today
```

и:

```
```python id="pguhmo" delivered_at
```

Поэтому моя основная гипотеза состоит в том, что проблема находится на границе загрузки legacy-данных: историческая запись не нормализуется в тот же тип, который ожидает `RefundService`.

Это пока гипотеза. Теперь её нужно подтвердить экспериментом.

## 15. Четвёртый промпт: проверяем гипотезы фактически

Отправляю Hermes:

```
```text id="hnjccx"
```

Теперь выполни диагностический план.

Файлы пока не изменяй.

Разрешено:

читать исходники;

запускать `final_debug_lab.reproduce_bug`;

выполнять безопасные Python-команды для просмотра фактических значений и типов;

запускать существующие тесты.

Запрещено:

изменять production-код;

изменять тесты;

создавать исправление;

маскировать исключение через `try/except`.

Проверяй гипотезы начиная с самой приоритетной.

Для каждого выполненного эксперимента зафиксируй:

какую гипотезу проверяли;

какую команду выполнили;

что ожидали увидеть;

что реально получили;

подтвердилась гипотеза, ослабла или была опровергнута.

После того как одна гипотеза будет достаточно подтверждена, не переходи автоматически к исправ-

лению.

Сначала дай итог диагностики.

Теперь агент должен использовать terminal, а не только рассуждение.

16. Какую проверку я ожидаю увидеть

Одну из самых простых проверок можно выполнить так:

```
```powershell id="1buh05" ..venv\Scripts\python.exe -c "from final_debug_lab.legacy_orders import get_order; print(type(get_order(1001)['delivered_at'])); print(type(get_order(1002)['delivered_at']))"
```

Результат должен показать разные типы.

Для одного заказа:

```
```text id="t13l6s"
<class 'datetime.date'>
```

Для другого:

```
```text id="igy5tp"
```

Теперь у нас уже есть прямое подтверждение различия.

## 17. Проверяем, действительно ли это объясняет исключение

Можно дополнительно выполнить:

```
```powershell id="pkzli1"
.\.venv\Scripts\python.exe -c "from datetime import date; from final_debug_lab.legacy_orders import get_order; order = get_order(1002); print(date(2026, 9, 20) - order['delivered_at'])"
```

Эта команда должна воспроизвести тот же тип ошибки, который присутствует в stack trace.

Теперь связь стала значительно сильнее.

Мы не просто заметили необычное значение.

Мы показали, что именно оно воспроизводит наблюдаемый технический сбой.

18. Формулируем root cause

После проверки я формулирую причину самостоятельно.

Непосредственная ошибка возникает в `RefundService`, когда выполняется операция с датами.

Но сам `RefundService` корректно работает с заказом 1001, где `delivered_at` представлен объектом `date`.

Следовательно, первопричина находится раньше.

Legacy-источник возвращает поле `delivered_at` в виде строки, тогда как остальная система ожидает нормализованное значение даты.

Таким образом, строка в `RefundService` является местом проявления ошибки, а не местом возникновения некорректного состояния данных.

19. Пятый промпт: просим Hermes подтвердить root cause

Теперь отправляю:

```
```text id="o0k7lb" На основании только фактически выполненных проверок сформулируй root cause.
```

Раздели:

симптом;

непосредственное исключение;

место проявления;

первопричину.

Отдельно объясни, почему изменение `RefundService` с целью просто принять строку не обязательно является лучшим первым исправлением.

Определи, на какой границе системы логичнее нормализовать legacy-данные.

Пока не изменяй код.

Здесь я проверяю архитектурное мышление агента.

`RefundService` работает с нормальной моделью заказа.

Значит, логичнее привести legacy-данные к нормальному представлению при их загрузке, а не заставлять бизнес-сервис поддерживать разные внутренние типы одного поля.

## 20. Перед исправлением создаём regression-тест

Теперь возвращаюсь к принципу предыдущей практики.

Я не хочу сначала исправить код, а потом придумать тест, который подтвердит уже готовое решение.

Сначала создаём проверку, которая воспроизводит дефект.

Отправляю Hermes:

```
```text id="p35naz"
```

Теперь создай regression-тест для подтверждённого BUG-001.

Создай:

```
tests/final_debug_lab/test_legacy_order_bug.py
```

Тест должен проверять фактический пользовательский сценарий.

Используй настоящий:

```
get_order(1002)
```

настоящий:

```
RefundService
```

и настоящий:

```
ReturnRepository
```

с SQLite in-memory.

Не подменяй get_order mock-объектом.

Сценарий:

загрузить заказ 1002;

создать RefundService;

попытаться создать возврат одной единицы product_id 101 на дату 2026-09-20;

ожидать успешное создание заявки;

проверить order_id;

проверить amount;

проверить status = "requested".

Production-код пока не исправляй.

После создания теста запусти только этот файл:

```
.\.venv\Scripts\python.exe -m pytest tests\final_debug_lab\test_legacy_order_bug.py -v
```

Мы ожидаем RED.

Если тест неожиданно проходит, не изменяй его автоматически. Сначала объясни почему.

Теперь Hermes должен создать воспроизводимый regression-тест.

21. Проверяем RED самостоятельно

После генерации запускаю:

```
```powershell id="h45t2o" ..venv\Scripts\python.exe -m pytest tests\final_debug_lab\
test_legacy_order_bug.py -v
```

Тест должен упасть.

Это хороший результат.

Мы доказали, что дефект теперь воспроизводится не только вручную, но и автоматизированной проверкой.

## 22. Проверяем, что тест действительно описывает баг

Открываю:

```
```text id="k4hauc"
tests\final_debug_lab\test_legacy_order_bug.py
```

Мне важно, чтобы тест не содержал:

```
```python id="fd4xrv" pytest.raises(TypeError)
```

Такой тест закрепил бы существующую ошибку как правильное поведение.

Наш баг-репорт говорит, что операция должна быть успешной.

Следовательно, regression-тест должен ожидать успешный возврат, а падать он должен из-за текущего дефекта production-кода.

## 23. Теперь разрешаем минимальное исправление

Отправляю Hermes:

```
```text id="3kgi1s"
Теперь исправь подтверждённую первопричину BUG-001.
```

Разрешено изменять только:

```
final_debug_lab/legacy_orders.py
```

Цель:

внешний пользователь `get_order` должен получать `delivered_at` в одном и том же типе независимо от источника записи.

Если `delivered_at` уже является `datetime.date`, сохрани его без изменения.

Если `delivered_at` является ISO-строкой формата YYYY-MM-DD, преобразуй её через стандартный Python API в `datetime.date`.

Не изменяй:

```
testing_lab/refunds.py;
```

существующие тесты `testing_lab`;

```
BUG_REPORT.md;
```

```
reproduce_bug.py.
```

Не добавляй сторонние библиотеки.

После изменения сначала запусти:

```
.\.venv\Scripts\python.exe -m pytest tests\final_debug_lab\test_legacy_order_bug.py -v
```

Затем запусти:

```
.\.venv\Scripts\python.exe -m pytest tests\testing_lab tests\final_debug_lab -v
```

После этого остановись и сообщи фактические результаты.

Это уже controlled fix.

24. Какое минимальное изменение я ожидаю

После исправления `get_order()` может выглядеть так:

```
python id="6z7z8v" def get_order(order_id: int) -> dict: if order_id not in _ORDERS: raise KeyError(f"Заказ {order_id} не найден")
```

```
order = deepcopy(_ORDERS[order_id])

delivered_at = order.get("delivered_at")

if isinstance(
```

```
        delivered_at,
        str,
    ):
        order["delivered_at"] = date.fromisoformat(delivered_at)

    return order
```

Здесь я исправляю данные на границе их загрузки.

Бизнес-логика `RefundService` остаётся одинаковой для новых и legacy-заказов.

25. Проверяем regression-тест

Самостоятельно запускаю:

```
```powershell id="8qe66b"
.\.venv\Scripts\python.exe -m pytest tests\final_debug_lab\test_legacy_order_bug.py -v
```

Теперь ожидаю:

```
```text id="z2r1g3" passed
```

Дальше запускаю весь связанный набор:

```
```powershell id="1ia23p"
.\.venv\Scripts\python.exe -m pytest tests\testing_lab tests\final_debug_lab -v
```

Мне важно убедиться, что исправление BUG-001 не сломало предыдущую функциональность.

## 26. Повторно запускаем исходное воспроизведение

Теперь выполняю первоначальную команду:

```
```powershell id="583jzp" ..venv\Scripts\python.exe -m final_debug_lab.reproduce_bug
```

Сохраняю новый журнал отдельно и открываю его:

```
```powershell id="130t6m"
Copy-Item .\final_debug_lab\logs\app.log .\final_debug_lab\logs\after_fix.log -Force
Get-Content .\final_debug_lab\logs\after_fix.log
```

Теперь оба `request_id` должны завершиться успешным созданием возврата.

Для:

```
```text id="i6nmvq" REQ-OK-1001
```

поведение не изменилось.

Для:

```
```text id="71r6xy"  
REQ-BUG-1002
```

ошибка должна исчезнуть.

Это уже end-to-end подтверждение исправления внутри нашего учебного сценария.

## 27. Сравниваем старый и новый результат

Я отдельно объясняю студентам, почему одного зелёного regression-теста было бы недостаточно для полноценного расследования.

У нас был исходный баг-репорт.

Был оригинальный stack trace.

Мы воспроизвели ошибку отдельно.

Создали автоматизированный regression-тест.

Исправили подтверждённую первопричину.

Повторно запустили regression-тест.

Запустили существующие тесты.

После этого ещё раз выполнили исходный пользовательский сценарий и сохранили новый лог отдельно от исходного свидетельства.

Именно совокупность этих данных позволяет мне уверенно сказать, что конкретный инцидент устранён.

Для прямого сравнения открываю оба снимка:

```
```powershell id="k9f4se" Get-Content .\final_debug_lab\logs\before_fix.log Get-Content .\final_debug_lab\  
logs\after_fix.log
```

В первом снимке должен присутствовать `TypeError` для `REQ-BUG-1002`, а во втором – два события `возврат\_создан` без `ERROR` и `Traceback`.

28. Просим Hermes сформировать итоговый incident report

Теперь создаём последний технический документ курса.

Отправляю Hermes:

```
```text id="an2p17"
```

Сформируй итоговый отчёт по BUG-001.

Создай:

final\_debug\_lab/INCIDENT\_REPORT.md

Используй только факты, подтверждённые в текущем расследовании.

Для сравнения результатов используй сохранённые журналы:

final\_debug\_lab/logs/before\_fix.log

final\_debug\_lab/logs/after\_fix.log

Документ должен содержать:

краткое описание симптома;

условия воспроизведения;

идентификатор проблемного запроса;

основное исключение;

место проявления ошибки;

подтверждённую первопричину;

какие гипотезы рассматривались;

какой эксперимент подтвердил основную гипотезу;

какой regression-тест был создан;

какое минимальное изменение внесено;

результаты тестов после исправления;

результат повторного воспроизведения;

почему выбранное исправление находится на границе загрузки данных, а не внутри RefundService.

Не добавляй неподтверждённые предположения.

Не утверждай, что проблема полностью невозможна во всех других частях системы.

Документируй только расследованный BUG-001.

После создания не изменяй больше никакие файлы.

Теперь у нас появляется законченный результат инженерного расследования.

## 29. Проверяем итоговый отчёт

Открываю:

```
```powershell id="dwb7yt" Get-Content .\final_debug_lab\INCIDENT_REPORT.md
```

Я проверяю, чтобы Hermes не перепутал:

```
```text id="8h5fko"
место падения
```

и:

```
```text id="utbgue" root cause
```

Это одна из главных целей сегодняшнего занятия.

В отчёте должно быть видно, что исключение возникло внутри расчёта срока возврата, но некорректное состояние появилось раньше – при загрузке legacy-данных.

30. Смотрим итоговый Git diff

Выполняю:

```
```powershell id="o1g045"
git status --short -- .gitignore final_debug_lab tests\final_debug_lab
```

Затем:

```
```powershell id="s1jg7n" git diff -- .gitignore final_debug_lab tests\final_debug_lab
```

Я ожидаю увидеть изменение:

```
```text id="o0avg1"
final_debug_lab\legacy_orders.py
```

новый regression-тест:

```
```text id="ksc0qe" tests\final_debug_lab\test_legacy_order_bug.py
```

и итоговый документ:

```
```text id="21os5h"
final_debug_lab\INCIDENT_REPORT.md
```

Сам `RefundService` изменяться не должен.

## 31. Просим Hermes провести финальную самопроверку

Последний промпт агенту в рамках курса:

```
```text id="39ydqh" Проведи финальную проверку BUG-001.
```

Ничего не изменяй.

Проверь изменения `.gitignore`, `final_debug_lab` и `tests/final_debug_lab`.

Убедись, что исправление ограничено подтверждённой первопричиной.

Запусти:

```
..env\Scripts\python.exe -m pytest tests\testing_lab tests\final_debug_lab -v
```

Затем запусти:

```
..env\Scripts\python.exe -m final_debug_lab.reproduce_bug
```

После этого прочитай:

```
final_debug_lab/logs/app.log
```

и:

```
final_debug_lab/logs/before_fix.log
```

и:

```
final_debug_lab/logs/after_fix.log
```

и:

```
final_debug_lab/INCIDENT_REPORT.md
```

Подготовь финальный вывод.

Отдельно укажи:

подтверждается ли исправление тестами;

подтверждается ли оно исходным сценарием воспроизведения;

остались ли ошибки в новом app.log;

подтверждается ли изменение сравнением журналов до и после;

были ли внесены несвязанные изменения;

соответствует ли INCIDENT_REPORT фактическим результатам.

Не исправляй ничего автоматически.

Это последний рабочий промпт курса.

32. Финальный commit

Если результаты правильные, фиксирую итог:

```
```powershell id="izw5ow"
git add .gitignore final_debug_lab__init__.py final_debug_lab\legacy_orders.py final_debug_lab\
reproduce_bug.py final_debug_lab\BUG_REPORT.md final_debug_lab\INCIDENT_REPORT.md tests\
final_debug_lab\test_legacy_order_bug.py
```

Проверяю:

```
```powershell id="c7lhcz" git diff --staged
```

После этого:

```
```powershell id="p1lrb1"
git commit -m "Практика 3.8-3.9: расследовать ошибку legacy-заказа"
```

Если репозиторий связан с удалённым, отправляю новый коммит обычной для проекта командой Git. Затем проверяю только область текущей практики:

```
```powershell id="lky6ds" git status --short -- .gitignore final_debug_lab tests\final_debug_lab
```

В перечисленных путях не должно оставаться незакоммиченных изменений. Посторонние файлы в репозитории не относятся к критериям завершения этой практики.

33. Что произошло с точки зрения анализа логов

Теперь я возвращаюсь к теме 3.8.

Мы начали не с исходного кода, а с наблюдаемого инцидента.

По `request\_id` разделили успешный и неуспешный сценарии.

Stack trace дал точное место технического падения.

Сравнение соседних событий показало различие входных данных.

После этого код позволил понять, почему именно это различие приводит к исключению.

Таким образом, лог использовался не просто как файл, в котором мы нашли слово `ERROR`. Мы восстановили конкретную последовательность обработки одного запроса.

34. Что произошло с точки зрения генерации гипотез

Теперь возвращаюсь к теме 3.9.

Мы не попросили Hermes сразу назвать одну причину.

Сначала агент должен был сформировать несколько проверяемых объяснений.

Для каждой версии мы потребовали указать подтверждающие данные, противоречия, способ проверки и ожидаемый результат.

Только после эксперимента одна из гипотез стала подтверждённой первопричиной.

Это принципиально отличается от подхода:

```
```text id="55yn5h"
```

AI сказал, что проблема здесь, поэтому исправляем.

Модель предложила направление исследования, а истинность гипотезы определил результат выполнения программы.

## 35. Почему мы не исправляли stack trace напрямую

В этой работе было особенно важно не остановиться на строке:

```
```python id="9i0ozk" today - delivered_at
```

Она действительно вызвала исключение.

Но если бы я начал исправлять `RefundService`, добавляя туда проверки строк, я бы смешал бизнес-логику с особенностями legacy-источника.

Мы исследовали путь данных и нашли место, где объект заказа должен был быть приведён к нормальному внутреннему представлению.

Поэтому исправление получилось небольшим и локализованным.

36. Почему regression-тест является частью исправления

Без regression-теста мы могли бы исправить баг сегодня, а через несколько недель снова получить ту же проблему.

Тест теперь фиксирует реальный сценарий:

```
```text id="clv04h"  
legacy-заказ 1002 должен успешно пройти через RefundService
```

Если кто-то снова уберёт нормализацию даты, test suite обнаружит изменение.

Так история реального дефекта превращается в новую автоматическую проверку проекта.

## 37. Чем Hermes помог в этой работе

---

Hermes здесь последовательно выполнял разные роли.

Сначала он был аналитиком логов.

Затем исследовал код.

После этого выступал генератором диагностических гипотез.

Дальше использовал terminal для проверки предположений.

После подтверждения root cause сгенерировал regression-тест.

Только затем получил разрешение на минимальное изменение production-кода.

В самом конце он подготовил технический incident report и провёл итоговую проверку.

Фактически доступные файловые и терминальные инструменты определяются конфигурацией среды. Для расследования я заранее проверил необходимые возможности и ограничивал действия агента содержанием каждого промпта.

## 38. Что принципиально оставалось за человеком

---

При всём объёме автоматизации я не передавал агенту окончательное решение автоматически.

Я определил, какие данные являются источниками истины.

Я запретил преждевременное исправление.

Я потребовал отделить факты от гипотез.

Я проверил, что эксперимент действительно воспроизводит проблему.

Я определил допустимую область изменения.

Я посмотрел Git diff перед commit.

---

Это и есть правильная роль человека в agent-assisted development: AI ускоряет исследование и выполнение технических операций, но инженер сохраняет контроль над критериями доказательства и изменением системы.

## 39. Итог заключительной практики

---

В начале занятия у нас было только сообщение:

```
text id="x49ggg" Некоторые старые заказы завершаются ошибкой при создании возврата.
```

К концу практики у нас есть подтверждённая причинная цепочка, воспроизводимый сценарий, regression-тест, минимальное исправление и документированный результат расследования.

Мы не определяли root cause по одному stack trace.

Мы не принимали первую гипотезу модели за факт.

Мы не исправляли тест, чтобы он стал зелёным.

Мы не изменяли код до воспроизведения проблемы.

И мы не считали работу завершённой только потому, что Hermes написал сообщение об успехе.

Финальное подтверждение получили фактическими командами, тестами, повторным воспроизведением и анализом нового лога.

## 40. Завершение курса

---

На этом практическая часть курса заканчивается.

Мы начали с использования AI как помощника и постепенно дошли до управляемой агентной разработки. В течение курса агент работал с контекстом проекта, ролями, Tools и Skills, внешними знаниями, Git и Pull Request, процессами, безопасностью, планированием спринта, промптами, code review, генерацией тестов, документацией и, наконец, диагностикой реального дефекта.

Заключительная практика специально объединяет эти навыки в одном процессе. Здесь уже недостаточно уметь написать хороший промпт или попросить модель сгенерировать код. Необходимо управлять тем, какие данные получает агент, какие действия ему разрешены, что считается доказательством, когда можно изменять production-код и как проверить результат после изменения.

Именно такой подход я считаю итоговой целью курса: использовать AI-агента не как генератор случайных решений, а как контролируемый инженерный инструмент, который работает внутри обычного процесса разработки и оставляет проверяемый технический результат.

---